

Đánh giá và cải tiến cơ chế phát hiện DNS Cache Poisoning dựa trên mô hình phòng thủ POPS

Nguyễn Minh Phúc Khang^{1,2}, Trang Tuấn Anh^{1,2}, and Nghi Hoàng Khoa^{1,2}

¹ Khoa Mạng máy tính và Truyền thông

² Trường Đại học Công nghệ Thông tin, Đại học Quốc gia Thành phố Hồ Chí Minh
{24520758, 24520131}@gm.uit.edu.vn khoanh@uit.edu.vn

Abstract. Đề tài tập trung triển khai, đánh giá và mở rộng module *Detection* của POPS – một hệ thống phòng thủ có cấu trúc trước tấn công đầu độc bộ nhớ đệm DNS (DNS Cache Poisoning). Nhóm thực hiện xây dựng môi trường Docker có kiểm soát nhằm mô phỏng ba nhóm tấn công kinh điển gồm S-type, fragmentation-based và out-of-bailiwick, từ đó triển khai và đánh giá các rule phát hiện tương ứng của POPS. Hiệu quả của hệ thống được đo thông qua Tỷ lệ tấn công thành công (Attack Success Rate – ASR) và độ trễ xử lý (Latency). Kết quả thực nghiệm cho thấy các rule phòng thủ có khả năng ngăn dữ liệu poisoning đi vào cache trong các kịch bản mô phỏng, đưa ASR về 0.00% khi được kích hoạt. Bên cạnh đó, đề tài đề xuất phiên bản *Rl₂-fragment detection* cải tiến dựa trên entropy IPID nhằm phân biệt tốt hơn giữa fragment hợp lệ và SFragment flood, cách tiếp cận hướng đến việc làm giảm false positive so với cách xử lý chặn fragment thận trọng thông thường.

Keywords: DNS Cache Poisoning · POPS · IPID Entropy

1 Giới thiệu

Hệ thống phân giải tên miền (Domain Name System – DNS) là thành phần nền tảng của Internet, có nhiệm vụ chuyển đổi tên miền sang địa chỉ IP. Do cấu hình truyền thống dựa trên UDP thiếu cơ chế xác thực mặc định, resolver có thể trở thành mục tiêu của tấn công Đầu độc bộ nhớ đệm DNS (DNS Cache Poisoning) nhằm điều hướng đến máy chủ giả mạo, tạo điều kiện cho phishing, đánh cắp thông tin hoặc phát tán mã độc. Mức độ nghiêm trọng của rủi ro này được phản ánh trong các báo cáo gần đây: WEF ghi nhận *cyber-enabled fraud and phishing* là một trong những mối quan ngại hàng đầu của CEO năm 2026 [8], trong khi Forrester Consulting và EfficientIP cho thấy 95% tổ chức từng gặp sự cố liên quan đến DNS với thiệt hại trung bình khoảng 942,000 USD mỗi sự cố [3].

Bên cạnh các tấn công brute-force TXID/source port truyền thống, DNS Cache Poisoning ngày càng xuất hiện nhiều biến thể phức tạp hơn. Theo POPS, attacker có thể lợi dụng IP fragmentation thông qua S_{Frag} và B_{Frag} , hoặc khai thác các bản ghi ngoài phạm vi thẩm quyền (*out-of-bailiwick* – OoB) trong tấn công S_{OoB} . Các kỹ thuật này không chỉ phụ thuộc vào việc đoán định danh DNS, mà còn tận dụng cách resolver xử lý fragment và kiểm tra *bailiwick*. Do

đó, những IDS truyền thống như Suricata hoặc Snort có thể gặp hạn chế khi phát hiện các biến thể DNS Cache Poisoning cần ngữ cảnh sâu hơn [1].

Trong bối cảnh đó, Afek và cộng sự đã đề xuất POPS (*Poisoning Prevention System*) tại USENIX Security 2025 như một cơ chế phòng thủ có thể tích hợp vào hệ thống IPS [1]. POPS không chỉ kiểm tra từng gói DNS riêng lẻ, mà còn theo dõi hành vi truy vấn và phản hồi theo thời gian để phát hiện các dấu hiệu bất thường. Tuy nhiên, một số cơ chế xử lý của POPS vẫn tồn tại sự đánh đổi giữa hiệu quả phòng thủ và tính sẵn sàng của dịch vụ, đặc biệt trong các trường hợp liên quan đến phân mảnh IP và khả năng hỗ trợ TCP của resolver.

Xuất phát từ vấn đề trên, đề tài có ba đóng góp chính: (i) xây dựng môi trường thực nghiệm có kiểm soát để tái hiện các nhóm tấn công DNS Cache Poisoning kinh điển (S-type, fragmentation-based và OoB); (ii) đánh giá khả năng phát hiện của các rule Rl_1 , Rl_2 và Rl_3 theo hướng tiếp cận POPS; và (iii) đề xuất phiên bản Rl_2 cải tiến dựa trên entropy IPID nhằm giảm hạn chế của cách xử lý cực đoan đối với DNS fragmentation, hướng đến cân bằng tốt hơn giữa ngăn chặn tấn công và duy trì tính sẵn sàng của dịch vụ DNS.

2 Các nghiên cứu liên quan

Các công trình trước đó cho thấy IDS truyền thống như Snort và Suricata chủ yếu dựa trên tập luật/chữ ký và thường theo dõi lưu lượng ở mức gói tin, luồng hoặc địa chỉ IP. Cách tiếp cận này phù hợp với các dấu hiệu DNS bất thường tổng quát, nhưng còn hạn chế khi cần theo dõi trạng thái truy vấn–phản hồi theo từng tên miền hoặc kiểm tra ngữ cảnh sâu như phân mảnh IP và quan hệ *bailiwick*. Ngược lại, POPS cung cấp hướng tiếp cận có trạng thái và theo ngữ cảnh domain rõ ràng hơn, bằng cách ánh xạ các biến thể DNS Cache Poisoning thành ba nhóm hành vi và rule phát hiện tương ứng [1] (Bảng 1). Tuy nhiên, nghiên cứu tiền đề chủ yếu tập trung vào thiết kế tổng thể và đánh giá trong môi trường mô phỏng. Do đó, việc tái hiện riêng từng rule của POPS trong từng lab độc lập, xây dựng kịch bản tấn công tương ứng và đo lường hiệu quả theo từng nhóm kỹ thuật vẫn cần được kiểm chứng thêm.

Bên cạnh ưu điểm phát hiện theo ngữ cảnh, POPS vẫn tồn tại hạn chế về tính sẵn sàng khi xử lý DNS fragmentation. Cụ thể, Rl_2 -*fragment detection* của POPS xem toàn bộ DNS response bị phân mảnh là bất thường và áp dụng cơ chế TC=1 để buộc resolver truy vấn lại qua TCP. Cách này giúp hạn chế dữ liệu độc đi vào cache, nhưng có thể ảnh hưởng đến các resolver không hỗ trợ tốt TCP fallback. Nghiên cứu ghi nhận khoảng 2.67% resolver không gửi lại truy vấn TCP sau khi nhận response có cờ TC, dẫn đến nguy cơ không phân giải được tên miền khi cơ chế này được áp dụng [1].

Từ khoảng trống đó, đề tài giới hạn phạm vi ở việc tái hiện và đánh giá module Phát hiện của POPS, bao gồm các luật Rl_1 , Rl_2 và Rl_3 tương ứng với ba nhóm tấn công chính. Trên cơ sở kết quả thực nghiệm, đề tài đề xuất một phiên bản Rl_2 cải tiến dựa trên entropy IPID nhằm giảm cách xử lý cực đoan đối với DNS fragmentation. Hướng tiếp cận này không chặn mặc định mọi fragment sau, mà đánh giá mức độ bất thường của phân bố IPID để cân bằng tốt hơn

Table 1. So sánh các hướng tiếp cận trong phòng thủ DNS Cache Poisoning.

Hướng tiếp cận	Phạm vi tiếp cận	Cơ chế chính	Nhận xét
IDS truyền thống	Theo dõi lưu lượng ở mức gói tin, luồng kết nối hoặc địa chỉ IP nguồn/đích. Phát hiện thường dựa trên các đặc trưng traffic cục bộ như mẫu gói, tần suất xuất hiện hoặc thống kê theo IP, thay vì duy trì trạng thái truy vấn-phản hồi DNS theo từng tên miền.	Sử dụng tập luật và chữ ký để so khớp các mẫu bất thường đã được định nghĩa trước trong lưu lượng mạng. Với DNS, các IDS như Snort và Suricata có thể phân tích một số trường ở tầng ứng dụng như truy vấn, phản hồi, mã trạng thái, số lượng bản ghi tài nguyên, kiểu bản ghi và các mẫu gói giống DNS spoofing [2,4,5,6,7].	Có hiệu quả với các dấu hiệu DNS bất thường tổng quát, nhưng hạn chế khi cần theo dõi trạng thái theo từng domain. Do tổng hợp theo IP hoặc ngưỡng số lượng gói, cơ chế này kém nhạy với brute-force TXID/source port phân tán theo tên miền. Cách tiếp cận này khó xử lý đầy đủ các biến thể cản ngữ cảnh sâu như IP fragmentation và OoB [1].
POPS	Quan sát thụ động các DNS response trên đường truyền và tổng hợp trạng thái theo tên miền trong một cửa sổ thời gian ngắn. Phạm vi phát hiện tập trung vào mối quan hệ giữa response, tên miền được truy vấn và dữ liệu được đưa vào cache.	Phân loại DNS Cache Poisoning thành ba nhóm hành vi chính: brute-force TXID/source port, khai thác IP fragmentation và vi phạm <i>bailiwick</i> . POPS sử dụng ba rule Rl_1 , Rl_2 , Rl_3 để phát hiện các nhóm này. Khi response bị đánh dấu nghi ngờ, hệ thống xóa nội dung Answer, Authority và Additional, sau đó đặt cờ TC=1 để buộc resolver truy vấn lại qua TCP [1].	Cung cấp cơ chế có trạng thái và theo ngữ cảnh domain, phù hợp hơn với các biến thể poisoning cần quan sát nhiều response trong thời gian ngắn. Cơ chế mitigation bằng Truncated giúp hạn chế dữ liệu độc đi vào cache mà vẫn cho phép resolver lấy lại câu trả lời hợp lệ qua TCP. Tuy nhiên, hệ thống phụ thuộc vào vị trí quan sát DNS response và có thể làm tăng số truy vấn TCP khi nhiều response bị đánh dấu nghi ngờ [1].

giữa khả năng phát hiện tấn công và hạn chế ảnh hưởng đến các luồng fragment hợp lệ.

3 Phương pháp thực hiện

Hệ thống thực nghiệm được xây dựng trên Docker Compose nhằm tạo ra một môi trường mô phỏng cô lập, có thể kiểm soát và tái hiện các luồng truy vấn DNS phục vụ đánh giá DNS Cache Poisoning. Trong mỗi lab, các container được đặt trong một Docker network riêng có dạng địa chỉ `10.x.0.0/24`, trong đó giá trị `x` thay đổi tùy theo từng nhóm thí nghiệm. Cách tổ chức này giúp các kịch bản được chạy độc lập, hạn chế ảnh hưởng chéo giữa các loại tấn công và bảo đảm kết quả đo có khả năng lặp lại.

3.1 Tổng quan kiến trúc hệ thống

Mỗi môi trường thí nghiệm gồm bốn thành phần chính: client, DNS resolver, authoritative DNS server và attacker được minh họa trong Hình 1. Client có nhiệm vụ sinh truy vấn DNS và ghi nhận kết quả phân giải. DNS resolver là thành phần trung tâm, đảm nhiệm việc xử lý truy vấn đệ quy, lưu DNS cache và thực thi các rule phòng thủ khi được kích hoạt. Khi bản ghi chưa tồn tại trong

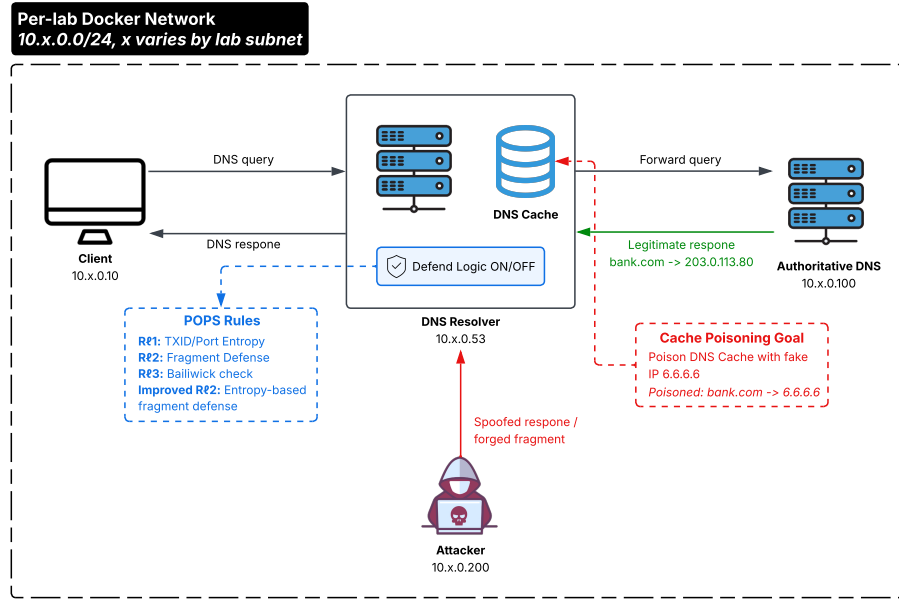


Fig. 1. Mô hình triển khai môi trường thực nghiệm trong Docker Compose.

cache, resolver chuyển tiếp truy vấn đến authoritative DNS server để nhận phản hồi hợp lệ. Authoritative DNS server được cấu hình để trả về bản ghi đúng cho tên miền mục tiêu trong phạm vi thí nghiệm.

Attacker được đặt trong cùng mạng thí nghiệm để gửi các phản hồi DNS giả mạo hoặc fragment giả hướng về resolver. Trong cấu hình của lab, authoritative DNS server trả về bản ghi hợp lệ cho **bank.com** với địa chỉ 203.0.113.80, trong khi mục tiêu của attacker là làm cho resolver lưu vào cache bản ghi giả khiến **bank.com** phân giải về 6.6.6.6. Quá trình này diễn ra trong cửa sổ thời gian trước khi phản hồi hợp lệ được xử lý, từ đó mô phỏng hành vi đầu độc DNS cache.

Các rule phòng thủ được tích hợp trực tiếp vào resolver và có thể bật/tắt theo từng kịch bản thực nghiệm. Nhờ đó, cùng một môi trường lab có thể được dùng để so sánh giữa trạng thái không phòng thủ, trạng thái có attacker nhưng chưa bật rule, và trạng thái có kích hoạt rule tương ứng. Cách thiết kế này giúp đánh giá rõ hơn vai trò của từng rule trong việc ngăn dữ liệu giả mạo đi vào DNS cache.

3.2 Thiết kế các kịch bản tấn công

Dựa trên môi trường thực nghiệm đã mô tả ở phần trước, đề tài thiết kế các kịch bản tấn công nhằm tái hiện ba nhóm DNS Cache Poisoning chính: S-type, fragmentation-based và OoB. Các nhóm kịch bản được tách thành những

lab riêng, nhưng vẫn dùng chung kiến trúc client–resolver–authoritative server–attacker, giúp cô lập tham số đặc thù của từng kỹ thuật và đánh giá riêng hiệu quả của rule phòng thủ tương ứng.

Nhóm S-type Nhóm S-type mô phỏng các tấn công brute-force dựa trên TXID và source port. Trong thực tế, nếu attacker không biết cả hai trường này, không gian đoán có thể lên đến 2^{32} ; nếu một trường đã bị cố định hoặc suy đoán được, không gian còn khoảng 2^{16} . Trong môi trường thực nghiệm, không gian tìm kiếm được thu nhỏ có kiểm soát để tái hiện attack trong thời gian ngắn. Mỗi biến thể chạy 150 truy vấn độc lập và attacker gửi tối đa 200 response giả trong mỗi cửa sổ truy vấn.

Với *TXID brute-force*, qname là **bank.com**, source port upstream của resolver được cố định tại 33333, còn TXID nằm trong dải 0..199. Attacker thắng race nếu một response giả có TXID khớp đến trước response hợp lệ.

Với *source-port brute-force*, attacker phải đoán đồng thời source port và TXID. Lab cấu hình source port trong dải 33300..33319 và TXID trong dải 0..9, tương ứng 20×10 tổ hợp cần thử trong mỗi cửa sổ truy vấn.

Với *Kaminsky-style*, client tạo cache miss bằng truy vấn tới subdomain không tồn tại trên hệ thống **victim.bank.com**. Resolver dùng source port cố định 33333 và TXID trong dải 0..199. Khác với brute-force trên một truy vấn cố định, biến thể này tận dụng nhiều cache miss độc lập; mỗi response giả vừa trả lời cho **victim.bank.com**, vừa chèn bản ghi độc cho **bank.com** vào additional section với địa chỉ 6.6.6.6. Nếu một response giả thắng race, resolver có thể lưu bản ghi độc của **bank.com** vào cache.

Nhóm Fragmentation-based Nhóm fragmentation-based mô phỏng các tấn công lợi dụng quá trình ghép mảnh IP của DNS response. Thay vì đoán trực tiếp TXID và source port, attacker gửi fragment thứ hai giả mạo chứa dữ liệu độc và cố làm fragment này được ghép với fragment đầu tiên hợp lệ từ authoritative server. Nếu hai fragment có cùng IPID trong cửa sổ xử lý, resolver có thể ghép dữ liệu giả vào response và lưu kết quả poisoning vào cache.

Với SFrag, IPID của response hợp lệ được chọn ngẫu nhiên, nên attacker phải brute-force không gian IPID. Trong cấu hình yếu hóa của lab, source port upstream được cố định tại 33333 và không gian IPID được thu nhỏ xuống 0..2047, tương ứng 2048 giá trị cần thử. Cấu hình này giúp tái hiện bản chất của SFrag mà không cần quét toàn bộ không gian IPID 16-bit.

Với BFrag, attacker được giả định đã biết hoặc suy ra IPID mục tiêu. Trong lab, IPID được cố định bằng 777 ở cả phía authoritative server và attacker. Do đó, attacker không cần quét toàn bộ không gian IPID mà chỉ cần liên tục gửi fragment giả với IPID đã biết, khiến khả năng poisoning phụ thuộc chủ yếu vào thời điểm fragment được resolver xử lý.

Nhóm OoB Nhóm OoB mô phỏng tấn công lợi dụng việc resolver xử lý không chặt chẽ các bản ghi nằm ngoài phạm vi thẩm quyền của authoritative server.

Client tạo các truy vấn mới thuộc zone hợp lệ **example.net** để kích hoạt cache miss, trong khi attacker gửi DNS response giả mạo trong cửa sổ race.

Response giả được thiết kế sao cho phần Answer phù hợp với truy vấn đang xử lý, còn Additional section chứa bản ghi ngoài bailiwick cho **bank.com**. Khi defense bị tắt, resolver có thể chấp nhận bản ghi này mà không kiểm tra phạm vi thẩm quyền, từ đó lưu dữ liệu poisoning vào cache.

Để tái hiện tấn công trong thời gian hữu hạn, lab sử dụng cấu hình entropy yếu: source port upstream của resolver được cố định tại 33333, còn TXID được thu nhỏ từ không gian 16-bit xuống dải 0..1023, tương ứng 1024 giá trị cần thử. Authoritative server trì hoãn phản hồi trong 0.25 giây để tạo cửa sổ race; trong thời gian này, attacker giả mạo IP nguồn của authoritative server và gửi lần lượt các response ứng với toàn bộ không gian TXID. Cấu hình này nhằm minh họa cơ chế SOoB và đánh giá tác động của rule kiểm tra bailiwick trong môi trường lab cô lập.

Bên cạnh cấu hình chính nêu trên, đề tài triển khai thêm một cấu hình phụ với entropy đầy đủ cho hai nhóm Fragmentation-based và OoB. Ở cấu hình này, source port upstream của resolver được chọn ngẫu nhiên, TXID giữ nguyên không gian 16-bit và IPID không bị thu nhỏ đối với các kịch bản fragmentation. Mục tiêu của cấu hình phụ là làm mốc so sánh với điều kiện resolver có entropy mạnh hơn. Qua đó, thí nghiệm có thể đối chiếu giữa trường hợp attacker được tạo điều kiện để tái hiện cơ chế poisoning trong lab và trường hợp không gian định danh đủ lớn khiến xác suất đoán đúng giảm đáng kể.

3.3 Triển khai cơ chế phòng thủ

Trong nghiên cứu của Afek và cộng sự [1], POPS cho thấy cách tiếp cận phòng thủ dựa trên việc nhận diện các mẫu hành vi chung của DNS Cache Poisoning thay vì viết chữ ký riêng cho từng biến thể. Kế thừa hướng tiếp cận này, đồ án tích hợp các rule phát hiện vào resolver để có thể bật/tắt theo từng kịch bản, từ đó so sánh trực tiếp trạng thái không phòng thủ và có phòng thủ. Bên cạnh việc tái hiện các rule cơ sở, đồ án đề xuất thêm một phiên bản Rl_2 cải tiến dựa trên entropy IPID nhằm giảm chặn nhằm đối với các luồng fragment hợp lệ.

Rl_1 : Phát hiện brute-force TXID và source port Rl_1 được dùng để phát hiện nhóm tấn công S-type, trong đó attacker gửi nhiều DNS response giả mạo trong một cửa sổ thời gian ngắn để brute-force TXID hoặc source port. Đặc trưng của nhóm này là các response cùng hướng tới một truy vấn mục tiêu, nhưng thay đổi ở các trường định danh cần đoán. Vì vậy, resolver gom các response theo từng truy vấn đang chờ và theo dõi số lượng response xuất hiện trong cửa sổ thời gian W . Gọi $N_q(W)$ là số response ứng với truy vấn q ; nếu $N_q(W) > \tau$ và các response có mẫu thay đổi tập trung ở TXID hoặc destination port, truy vấn đó được đánh dấu là đáng ngờ.

Rl_2 : Phòng thủ trước tấn công dựa trên fragmentation Rl_2 được dùng để phòng thủ trước các tấn công SFragment và BFragment, tức các biến thể không có

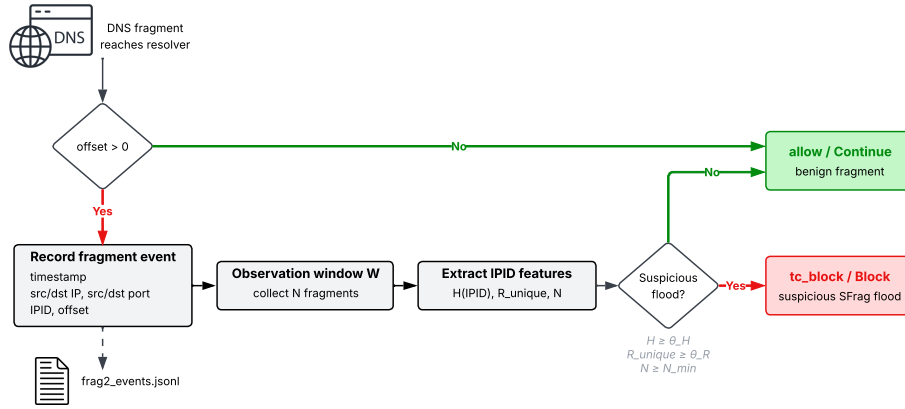


Fig. 2. Phát hiện SFrage flood bằng Rl_2 cải tiến dựa trên entropy IPID.

gắng đoán TXID hoặc source port trực tiếp mà lợi dụng cơ chế ghép mảnh IP của DNS response. Điểm nguy hiểm của nhóm này nằm ở chỗ fragment đầu tiên thường chứa các trường định danh quan trọng như TXID và port, còn fragment thứ hai có thể được attacker chuẩn bị sẵn với dữ liệu poisoning. Nếu fragment giả có IPID trùng với fragment hợp lệ, resolver có thể ghép hai mảnh này thành một response hoàn chỉnh và vô tình xử lý dữ liệu độc.

Vì vậy, Rl_2 xử lý DNS fragmentation theo hướng loại bỏ bề mặt tấn công từ các fragment sau. Khi phát hiện response bị phân mảnh, rule không cho các fragment có offset lớn hơn 0 tham gia vào quá trình ghép dữ liệu DNS. Cách xử lý này đặc biệt quan trọng với BFrage, vì attacker chỉ cần gửi một fragment thứ hai đã biết IPID mục tiêu là có thể tạo điều kiện cho poisoning nếu fragment đó được ghép với fragment đầu tiên hợp lệ. Bằng cách chặn hoặc vô hiệu hóa các fragment sau, Rl_2 làm mất khả năng attacker chèn dữ liệu độc thông qua IPID collision, từ đó giảm khả năng thành công của SFrage và BFrage.

Rl_3 : Kiểm tra nguyên tắc bailiwick Rl_3 được dùng để phát hiện các tấn công OoB - kiểu tấn công mà attacker lợi dụng trường Additional section của DNS response để chèn các bản ghi không thuộc phạm vi thẩm quyền của authoritative server. Khác với S-type hay fragmentation-based, điểm bất thường của SOoB không nằm ở số lượng response hay quá trình ghép mảnh, mà nằm ở quan hệ giữa tên miền được truy vấn và các bản ghi đi kèm trong response.

Với một response hợp lệ, resolver chỉ nên chấp nhận các bản ghi nằm trong phạm vi zone mà authoritative server có quyền trả lời. Do đó, Rl_3 kiểm tra các bản ghi trong Answer, Authority và Additional section trước khi cho phép lưu vào cache. Nếu response cho một truy vấn thuộc **example.net** lại cố chèn thêm bản ghi cho **bank.com**, bản ghi này được xem là nằm ngoài bailiwick và bị loại bỏ. Cách xử lý này ngăn attacker đưa dữ liệu poisoning vào cache thông qua các bản ghi bổ sung không thuộc thẩm quyền của server đang được truy vấn.

$R\ell_2$ cải tiến: Phát hiện SFrag dựa trên entropy IPID Mặc dù $R\ell_2$ gốc có thể ngăn chặn hiệu quả SFrag và BFrag bằng cách xử lý thận trọng qua việc xem DNS fragmentation là tín hiệu nguy hiểm và buộc resolver chuyển sang TCP bằng cờ TC=1. Cách tiếp cận này giúp giảm rủi ro poisoning, nhưng có thể ảnh hưởng đến các luồng fragment hợp lệ và làm một số resolver không phân giải được tên miền trong vài trường hợp. Từ hạn chế đó, đề án đề xuất phiên bản $R\ell_2$ cải tiến, trong đó resolver không chặn mặc định mọi fragment mà đánh giá mức độ bất thường của phân bố IPID trong một cửa sổ quan sát ngắn (Hình 2).

Trong phiên bản cải tiến, resolver ghi nhận các fragment có offset lớn hơn 0, gồm địa chỉ nguồn, địa chỉ đích, source port, destination port, IPID và offset. Các sự kiện này được lưu dưới dạng JSON và dùng để tính entropy IPID cùng tỷ lệ IPID duy nhất. Ý tưởng chính là SFrag flood thường tạo nhiều fragment giả với các IPID khác nhau để tăng xác suất trùng với fragment hợp lệ, khiến phân bố IPID phân tán hơn so với luồng fragment bình thường. Đề án sử dụng entropy Shannon để đo mức độ phân tán này:

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i,$$

trong đó X là tập các giá trị IPID quan sát được, còn p_i là xác suất xuất hiện của giá trị IPID thứ i . Bên cạnh đó, hệ thống xét thêm tỷ lệ IPID duy nhất:

$$R_{\text{unique}} = \frac{|\text{unique}(\text{IPID})|}{N},$$

với N là tổng số fragment trong cửa sổ quan sát. Khi số mẫu đủ lớn, entropy vượt ngưỡng và R_{unique} cao, resolver xem luồng này là SFrag flood và kích hoạt chặn. Ngược lại, các luồng có ít fragment hoặc phân bố IPID không bất thường sẽ được cho phép, nhằm giảm false positive so với $R\ell_2$ gốc.

4 Thử nghiệm và đánh giá

4.1 Thiết kế thực nghiệm

Các lab sử dụng cùng một quy trình thực nghiệm cơ bản: (1) client tạo truy vấn DNS nhằm kích hoạt cache miss tại resolver; (2) authoritative DNS server trả phản hồi hợp lệ sau một khoảng trễ có kiểm soát; (3) trong thời gian này, attacker gửi response giả mạo hoặc fragment giả để cố gắng làm resolver lưu kết quả sai vào cache. Cache của resolver được làm sạch giữa các vòng đo để mỗi lần thực hiện truy vấn của client đều độc lập với kết quả trước đó. Quy trình thực hiện được thiết kế nhằm so sánh hành vi của resolver trong ba trạng thái chính:

- **Baseline:** Attacker không được kích hoạt. Chế độ này dùng để kiểm tra trạng thái phân giải bình thường của resolver và ghi nhận độ trễ nền khi xảy ra cache miss.

- **Attack-off:** Attacker được kích hoạt nhưng rule phòng thủ tương ứng bị tắt, dùng để đo khả năng poisoning của từng kỹ thuật khi không có cơ chế bảo vệ.
- **Attack-on:** Attacker được kích hoạt và rule phòng thủ tương ứng được bật, dùng để đánh giá khả năng ngăn dữ liệu poisoning đi vào cache, đồng thời đối chiếu với *Attack-off* để xác định tác động của rule trước các kịch bản tấn công.

Mỗi kịch bản được thực hiện 150 lần truy vấn và được đánh giá thông qua các metrics:

- **Attack Success Rate (ASR):** tỷ lệ truy vấn bị poisoning, tức client nhận địa chỉ giả thay vì địa chỉ hợp lệ.
- **Latency trung bình (Avg. Lat.):** độ trễ trung bình của quá trình phân giải DNS, dùng để đánh giá chi phí hiệu năng tổng quát.
- **Latency p_{95} :** độ trễ tại phân vị 95, dùng để quan sát các trường hợp xử lý chậm hơn bình thường.

Riêng với Rl_2 cải tiến, nhóm bổ sung chế độ **Benign-on** để kiểm tra khả năng cho phép các luồng fragment hợp lệ khi rule entropy được bật, dùng để đánh giá liệu cơ chế cải tiến có giảm false positive so với Rl_2 gốc hay không. Bên cạnh đó, hệ thống ghi nhận thêm số fragment quan sát được, entropy IPID, tỷ lệ IPID duy nhất và số lần tc_block nhằm đánh giá khả năng ứng phó của hệ thống trong nhiều trường hợp khác nhau

4.2 Hiệu quả của các rule POPS trước các nhóm tấn công

Xác nhận trạng thái nền của môi trường thực nghiệm Bảng 2 cho thấy các query hợp lệ có ASR bằng 0.00%. Điều này phản ánh rằng resolver trả về bản ghi hợp lệ trong trạng thái bình thường, không xuất hiện poisoning ngoài ý muốn, nên các kết quả poisoning ở các thí nghiệm sau có thể được quy về tác động của attacker thay vì lỗi cấu hình của resolver hoặc cache. Về độ trễ, baseline có latency trung bình dao động từ 271.194 đến 320.214 ms, còn p_{95} nằm trong khoảng 271.535–294.206 ms. Kết quả này phản ánh chi phí nền của quá trình cache miss, đồng thời xác nhận hệ thống hoạt động ổn định trước khi đưa attacker và rule phòng thủ vào đánh giá.

Hiệu quả phòng thủ trước các kịch bản tấn công Cũng trong Bảng 2, khi attacker được kích hoạt nhưng rule phòng thủ bị tắt, mức độ tác động của từng nhóm tấn công có sự khác biệt rõ rệt thông qua giá trị ASR:

- **Nhóm S-type:** Hai biến thể brute-force riêng lẻ, gồm TXID brute-force và source-port brute-force, đều có ASR bằng 0.00%. Điều này cho thấy việc vét cạn một trường định danh là chưa đủ để poisoning thành công, vì attacker vừa phải đoán đúng giá trị cần thiết, vừa phải gửi response giả trước khi

Table 2. Kết quả thực nghiệm ở 3 chế độ Baseline, Attack-off và Attack-on.

Trạng thái	Kịch bản	Poisoned/Total	ASR	Avg. Lat. (ms)	p95 (ms)
Baseline	Normal query	0/150	0.00%	290.996	283.474
	TXID brute-force	0/150	0.00%	21.363	37.787
Attack-off	Source-port brute-force	0/150	0.00%	18.118	28.921
	Kaminsky-style	35/150	23.33%	20.117	31.485
	SFrag	147/150	98.00%	271.229	272.730
	BFrag	150/150	100.00%	280.248	273.955
	Out-of-Bailiwick	148/150	98.67%	260.647	376.623
	TXID brute-force	0/150	0.00%	21.958	36.979
Attack-on	Source-port brute-force	0/150	0.00%	16.803	29.399
	Kaminsky-style	0/150	0.00%	20.828	32.293
	SFrag	0/150	0.00%	275.347	274.763
	BFrag	0/150	0.00%	283.898	275.370
	Out-of-Bailiwick	0/150	0.00%	254.951	382.839

resolver nhận phản hồi hợp lệ. Với không gian đoán 2^{16} , xác suất khớp trong một cửa sổ race ngắn là rất thấp.

Ngược lại, Kaminsky-style đạt ASR 23.33% ở chế độ *Attack-off* do attacker liên tục tạo truy vấn con mới, qua đó tạo nhiều cache miss độc lập. Theo POPS, nếu mỗi cửa sổ race có $k = 200$ response giả và không gian đoán còn lại là $S = 2^{16}$, xác suất thắng một lần race xấp xỉ $P_{\text{hit}} \approx k/S = 200/65536 \approx 0.00305$. Sau $m = 200$ lần cache miss độc lập, xác suất thành công tích lũy là: $P_{\text{success}} = 1 - (1 - 0.00305)^{200} \approx 45.7\%$.

Giá trị thực nghiệm thấp hơn ước lượng do attacker không chỉ cần đoán đúng TXID/source port mà còn phải thắng về thời điểm gửi response. Kaminsky mạnh hơn brute-force thuần nhờ tạo nhiều lần thử độc lập, nhưng vẫn bị giới hạn bởi độ trễ, số response và thời điểm resolver nhận phản hồi hợp lệ.

- **Nhóm Fragmentation-based:** Hai biến thể SFrag và BFRag có mức tác động cao hơn rõ rệt so với brute-force định danh thông thường. Ở chế độ *Attack-off*, SFrag đạt ASR 98.00%, còn BFRag đạt ASR 100.00%. Kết quả này cho thấy khi attacker can thiệp được vào quá trình ghép mảnh, khả năng poisoning phụ thuộc chủ yếu vào việc tạo fragment giả có IPID phù hợp với fragment hợp lệ, thay vì chỉ dựa vào đoán TXID/source port.

Với BFRag, ASR đạt 100.00% do môi trường lab giả định attacker đã biết chính xác IPID mục tiêu. Khi đó, fragment giả chỉ cần dùng IPID trùng khớp để được ghép với fragment hợp lệ. Điều kiện này phản ánh trường hợp attacker có lợi thế mạnh, chẳng hạn khi IPID có thể được suy đoán từ hệ thống sinh tuần tự hoặc thông qua side-channel.

Với SFrag, attacker chưa biết IPID nên phải gửi nhiều fragment giả với các giá trị IPID khác nhau. Trong thí nghiệm, không gian IPID được thu nhỏ để tái hiện tấn công trong thời gian hữu hạn. Khi số fragment giả đủ lớn so với không gian cần quét, xác suất trùng IPID tăng dần đến ASR 98.00%.

- **Nhóm OoB:** Khác với S-type và Fragmentation-based, trọng tâm của OoB nằm ở việc lợi dụng các bản ghi ngoài phạm vi *bailiwick* trong phần Authority hoặc Additional. Kết quả cho thấy tỉ lệ ASR đạt 98.67% khi không bật rule

Table 3. Kết quả đánh giá cơ chế Rl_2 cải tiến dựa trên entropy IPID.

Case	ASR	Entropy avg	Decision	Avg. Lat. (ms)	p95 (ms)
Baseline	0.00%	N/A	N/A	314.476	288.744
Benign-on	0.00%	0.000	150 allow / 0 block	275.479	287.106
Attack-on	0.00%	10.836	0 allow / 148 block	304.422	279.118

phòng thủ. Điều này cho thấy nếu resolver không kiểm tra chặt chẽ phạm vi thẩm quyền của các bản ghi trong response, attacker có thể đưa dữ liệu poisoning vào cache với xác suất cao.

Đối với cấu hình entropy đầy đủ, các kịch bản Fragmentation-based và OoB đều ghi nhận ASR 0.00% sau 150 truy vấn thử nghiệm. Kết quả này cho thấy source port ngẫu nhiên, không gian TXID đầy đủ và IPID không bị thu nhỏ làm giảm đáng kể xác suất attacker đoán đúng các giá trị cần thiết trong số lượng truy vấn hữu hạn; không loại bỏ hoàn toàn rủi ro trong thực tế vì attacker vẫn có thể tận dụng các điều kiện bất lợi như IPID sinh theo quy luật, side-channel, khả năng gửi lượng lớn fragment giả hoặc resolver kiểm tra *bailiwick* chưa chặt chẽ. Do đó, entropy mạnh nên được xem là một lớp bảo vệ quan trọng, nhưng vẫn cần kết hợp với các rule phát hiện chuyên biệt cho fragmentation và OoB.

Khi bật các rule phòng thủ tương ứng, ASR của toàn bộ kịch bản trong chế độ *Attack-on* đều giảm về 0.00%, cho thấy các rule đã ngăn dữ liệu poisoning đi vào cache trong môi trường thực nghiệm. Sự chênh lệch rõ rệt giữa *Attack-off* và *Attack-on* cũng khẳng định vai trò của cơ chế phát hiện tại resolver, đặc biệt khi DNS Cache Poisoning không chỉ phụ thuộc vào brute-force TXID/source port mà còn có thể khai thác cơ chế xử lý fragment hoặc kiểm tra *bailiwick* chưa chặt chẽ. Nếu thiếu phòng thủ phù hợp, attacker có thể đầu độc cache DNS với tỷ lệ cao, từ đó tạo nguy cơ chuyển hướng người dùng đến hạ tầng giả mạo.

4.3 Hiệu quả của Rl_2 cải tiến trong phát hiện SFrag

Kết quả ở Bảng 3 cho thấy Rl_2 cải tiến có khả năng phân biệt giữa luồng fragment hợp lệ và SFrag flood. Ở *Baseline*, hệ thống phân giải DNS bình thường với ASR 0.00%; do không xuất hiện fragment có offset lớn hơn 0, các trường entropy và decision được ghi nhận là N/A.

Trong kịch bản *Benign-on*, entropy IPID trung bình bằng 0.000 và toàn bộ 150/150 decision đều là `allow`. Điều này cho thấy resolver có ghi nhận fragment và thực hiện đánh giá, nhưng phân bố IPID trong cửa sổ W không mang đặc trưng quét hàng loạt của SFrag flood. Vì vậy, rule không chặn thô mọi fragment như Rl_2 gốc, mà chỉ kích hoạt khi phân bố IPID vượt ngưỡng bất thường.

Ngược lại, trong *Attack-on*, entropy IPID trung bình đạt 10.836, vượt rõ rệt so với ngưỡng cấu hình 4.0. Kết quả này phản ánh đúng cơ chế của SFrag, trong đó attacker gửi nhiều fragment giả với các giá trị IPID khác nhau để tăng xác suất ghép với fragment hợp lệ. Việc toàn bộ 148/148 decision đều là `tc_block` cho thấy resolver đã nhận diện được SFrag flood dựa trên đặc trưng thống kê của IPID, thay vì chỉ dựa vào sự xuất hiện của fragment.

Về độ trễ, latency trung bình giữa các chế độ dao động từ 275.479 ms đến 314.476 ms, trong khi latency p_{95} nằm trong khoảng 279.118–288.744 ms. Mức chênh lệch này cho thấy việc ghi nhận fragment, tính entropy IPID và đưa ra quyết định `allow/tc_block` chưa tạo overhead đáng kể trong môi trường lab.

Sự khác biệt giữa *Benign-on* và *Attack-on* cho thấy cơ chế entropy-based Rl_2 vừa tránh chặn nhầm luồng fragment hợp lệ, vừa ngăn dữ liệu độc đi vào cache khi có SFrag flood. Dù vậy, kết quả này vẫn nằm trong phạm vi thí nghiệm có kiểm soát, khi triển khai thực tế, hiệu quả của cơ chế này cần được kiểm chứng thêm trên các kiểu lưu lượng, cách sinh IPID và điều kiện mạng đa dạng hơn.

4.4 Cân nhắc về đạo đức nghiên cứu

Đề tài chỉ phục vụ mục tiêu giáo dục, phân tích phòng thủ và đánh giá cơ chế phát hiện trong môi trường an toàn. Toàn bộ các kịch bản tấn công được triển khai trong môi trường Docker lab cô lập, chỉ sử dụng container, tên miền và dải địa chỉ nội bộ do nhóm thực hiện tự cấu hình. Các truy vấn, response và fragment giả mạo đều được tạo trong phạm vi thí nghiệm, không gửi đến hạ tầng DNS công cộng, hệ thống bên thứ ba hoặc người dùng thật. Các giá trị: `bank.com`, `203.0.113.80` và `6.6.6.6` chỉ đóng vai trò dữ liệu mô phỏng. Kết quả được trình bày ở mức tổng hợp thông qua ASR, latency và quyết định của rule, không công bố dữ liệu có thể ảnh hưởng đến hệ thống thực tế.

5 Kết luận

Đề án đã xây dựng một môi trường Docker có kiểm soát để mô phỏng và đánh giá các nhóm tấn công DNS Cache Poisoning kinh điển. Trên cơ sở đó, đề tài triển khai ba rule phòng thủ theo hướng tiếp cận POPS/DNS-CPM. Kết quả thực nghiệm cho thấy ASR chênh lệch rõ rệt giữa *Attack-off* và *Attack-on*. Một số biến thể phức tạp có thể gây poisoning với tỷ lệ đáng kể nếu không có phòng thủ thích hợp; trong khi đó khi bật rule tương ứng, ASR đều giảm về 0.00%, cho thấy các rule có vai trò quan trọng trong việc ngăn dữ liệu giả mạo đi vào cache. Bên cạnh các rule cơ sở, đề tài đề xuất phiên bản Rl_2 cải tiến dựa trên entropy IPID. Kết quả cho thấy cơ chế này phân biệt được giữa fragment hợp lệ và SFrag flood: ở *Benign-on*, entropy IPID bằng (0.000) nên toàn bộ decision là `allow`; trong khi ở *Attack-on*, entropy đạt 10.836 vượt ngưỡng 4.0, kéo theo toàn bộ decision là `tc_block`. Điều này cho thấy hướng tiếp cận dựa trên thống kê IPID có tiềm năng giảm false positive so với cách chặn toàn bộ fragment sau.

Mặc dù đã cố gắng hoàn thiện, đề tài vẫn còn một số hạn chế. Các thí nghiệm được thực hiện trong môi trường mô phỏng có kiểm soát, với không gian giá trị của các trường định danh được thu nhỏ để tái hiện tấn công trong thời gian hữu hạn. Bên cạnh đó, cơ chế entropy-based fragment detection vẫn phụ thuộc vào ngưỡng cấu hình thủ công và chưa bao phủ tốt các biến thể dùng IPID cố định hoặc cố tình giữ entropy thấp. Trong tương lai, hệ thống cần được đánh giá với IP fragmentation thực tế, các resolver phổ biến và lưu lượng DNS hợp lệ đa dạng hơn; đồng thời mở rộng Rl_2 bằng ngưỡng thích nghi, tốc độ packet và tương quan nguồn-đích.

Source code <https://github.com/0x-mpkane6/dns-poisoning-lab>

References

1. Afek, Y., Berger, H., Bremner-Barr, A.: POPS: From history to mitigation of DNS cache poisoning attacks. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 3537–3556. USENIX Association, Seattle, WA (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/afek>
2. Caswell, B., Beale, J., Baker, A.: Snort Intrusion Detection and Prevention Toolkit. Syngress (2007)
3. Forrester Consulting, EfficientIP: Opportunity snapshot: Embrace dns security and overcome hurdles of cloud adoption. Research report, EfficientIP (September 2025), <https://efficientip.com/resources/forrester-2025-dns-security-report/>, accessed: June 2026
4. Koziol, J.: Intrusion Detection with Snort. Sams Publishing (2003)
5. Open Information Security Foundation (OISF): Suricata: Open source network threat detection engine. <https://suricata.io/> (2024), accessed: June 2026
6. Suricata: Suricata emerging DNS events and rules keywords. <https://suricata.readthedocs.io/en/suricata-4.0.4/rules/dns-keywords.html> (2017), accessed: June 2026
7. Suricata: Suricata DNS events and EVE JSON output. <https://suricata.readthedocs.io/en/suricata-4.1.4/output/eve/eve-json-output.html#dns> (2018), accessed: June 2026
8. World Economic Forum, Accenture: Global cybersecurity outlook 2026. Global risk report, World Economic Forum (WEF) (January 2026), <https://www.weforum.org/publications/global-cybersecurity-outlook-2026/>, accessed: June 2026